# B2 — Segurança e Prevenção contra SQL Injection

## 1. Introdução

Nesta etapa do projeto Marketplace, realizamos testes de segurança para verificar se a aplicação desenvolvida em Java com Spring Boot estava protegida contra tentativas de SQL Injection.

O objetivo foi analisar como o sistema recebe os dados enviados pelo usuário, verificar a utilização de consultas parametrizadas e testar se uma entrada maliciosa conseguiria alterar o resultado de uma consulta ao banco de dados MySQL.

## 2. Análise do código

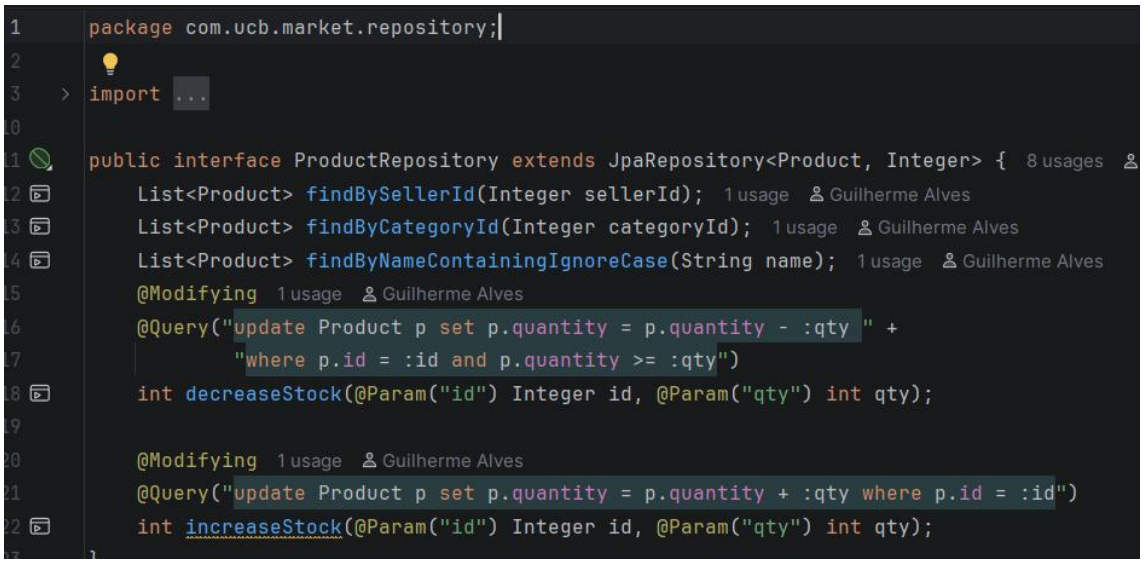
Inicialmente, analisamos o arquivo ProductRepository.java, responsável pelas operações de acesso aos dados dos produtos.

Foi identificado o método findByNameContainingIgnoreCase(String name), utilizado para pesquisar produtos pelo nome por meio do Spring Data JPA.

Também foram observadas consultas com @Query e @Param, que utilizam parâmetros para receber os valores das operações de atualização de estoque.

Essas práticas ajudam a evitar que entradas fornecidas pelo usuário sejam interpretadas como comandos SQL.

## Figura 1 — Código do ProductRepository.java



```
package com.ucb.market.repository;

import ...

public interface ProductRepository extends JpaRepository<Product, Integer> {  8 usages
    List<Product> findBySellerId(Integer sellerId);  1 usage  Guilherme Alves
    List<Product> findByCategoryId(Integer categoryId);  1 usage  Guilherme Alves
    List<Product> findByNameContainingIgnoreCase(String name);  1 usage  Guilherme Alves
    @Modifying  1 usage  Guilherme Alves
    @Query("update Product p set p.quantity = p.quantity - :qty " +
            "where p.id = :id and p.quantity >= :qty")
    int decreaseStock(@Param("id") Integer id, @Param("qty") int qty);

    @Modifying  1 usage  Guilherme Alves
    @Query("update Product p set p.quantity = p.quantity + :qty where p.id = :id")
    int increaseStock(@Param("id") Integer id, @Param("qty") int qty);
}
```

```java
package com.ucb.market.controller;

import com.ucb.market.dto.ProductRequest;
import com.ucb.market.dto.ProductResponse;
import com.ucb.market.services.ProductService;
import jakarta.validation.Valid;
import lombok.RequiredArgsConstructor;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/products")
@RequiredArgsConstructor
public class ProductController {

    private final ProductService productService;

    @GetMapping
    public List<ProductResponse> list(@RequestParam(required = false) Integer categoryId,
                                      @RequestParam(required = false) Integer sellerId,
                                      @RequestParam(required = false) String name) {
        return productService.list(categoryId, sellerId, name);
    }

    @GetMapping("/{id}")
    public ProductResponse findById(@PathVariable Integer id) {
        return productService.findById(id);
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public ProductResponse create(@RequestBody @Valid ProductRequest req) {
        return productService.create(req);
    }

    @PutMapping("/{id}")
    public ProductResponse update(@PathVariable Integer id, @RequestBody @Valid ProductRequest req) {
        return productService.update(id, req);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void delete(@PathVariable Integer id) {
        productService.delete(id);
    }
}
```

## 3. Testes realizados

Para verificar o comportamento da aplicação, executamos o projeto Spring Boot localmente e acessamos o endpoint /products pelo navegador.

Primeiramente, realizamos uma pesquisa normal utilizando o nome Samsung. A aplicação retornou dois produtos correspondentes à pesquisa, confirmando o funcionamento do filtro.

### Figura 2 — Pesquisa normal de produtos

```
0:
    id:             1
    name:           "Celular Samsung Galaxy A54"
    value:          1999.27
    quantity:       61
    sellerId:       4
    categoryId:     5
    categoryName:   "Celulares"
1:
    id:             42
    name:           "Celular Samsung Galaxy A54 Premium"
    value:          2019.01
    quantity:       4
    sellerId:       7
    categoryId:     5
    categoryName:   "Celulares"
```

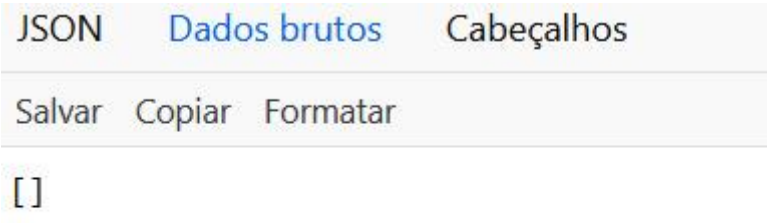

Em seguida, realizamos uma tentativa de SQL Injection utilizando a entrada:

' OR '1'='1

Essa expressão tenta criar uma condição sempre verdadeira, que poderia fazer uma consulta vulnerável retornar registros indevidamente.

No teste realizado, a aplicação retornou uma lista vazia ([]), sem apresentar todos os produtos cadastrados.

**Figura 3 — Tentativa de SQL Injection**




## 4. Verificação no banco de dados

Após o teste, utilizamos o MySQL Workbench para consultar a quantidade de produtos cadastrados no banco marketplace_db.

Foi executado o seguinte comando:

```
USE marketplace_db;


SELECT COUNT(*) AS total_produtos
FROM tb_product;
```

A consulta retornou 56 produtos cadastrados.

Esse resultado permitiu comparar a quantidade existente no banco com a resposta da aplicação durante a tentativa de SQL Injection.

### Figura 4 — Quantidade de produtos no banco de dados

```
1  USE marketplace_db;
2
3  SELECT COUNT(*) AS total_produtos
4  FROM tb_product;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| total_produtos |
| --- |
| 56 |

## 5. Proteção das credenciais

Também foi realizada uma alteração no arquivo application.properties para evitar que o usuário e a senha do banco de dados fossem escritos diretamente no código.

A configuração passou a utilizar as variáveis de ambiente DB_USERNAME e DB_PASSWORD, permitindo fornecer as credenciais durante a execução da aplicação.

Essa medida reduz o risco de exposição de novas credenciais nos arquivos do projeto. Entretanto, senhas anteriormente publicadas precisam ser substituídas, pois podem permanecer no histórico do repositório.

### Figura 5 — Configuração das credenciais

```
spring.datasource.username=${DB_USERNAME}
spring.datasource.password=${DB_PASSWORD}
```

## 6. Análise dos resultados

Os testes mostraram que a pesquisa normal retornou dois produtos, enquanto a tentativa de SQL Injection retornou uma lista vazia.

A consulta realizada no MySQL confirmou a existência de 56 produtos cadastrados, demonstrando que a entrada utilizada não conseguiu fazer a aplicação retornar todos esses registros.

Além disso, a análise do código identificou a utilização de recursos do Spring Data JPA e de parâmetros nas consultas.

Os resultados indicam que o endpoint de pesquisa resistiu à tentativa de SQL Injection realizada. Entretanto, esse teste isolado não garante que todas as funcionalidades da aplicação estejam livres de vulnerabilidades.

## 7. Conclusão

Com a realização desta atividade, foi possível compreender a importância das consultas parametrizadas e da proteção das credenciais no desenvolvimento de aplicações conectadas a bancos de dados.

A tentativa de SQL Injection não conseguiu alterar o comportamento esperado da pesquisa de produtos. Também foram identificadas práticas de acesso seguro aos dados e realizada uma melhoria na configuração das credenciais.

Dessa forma, o teste apresentou resultados favoráveis à segurança do endpoint analisado, reforçando a importância de prevenir vulnerabilidades durante o desenvolvimento do sistema.